Overcoming Arrow's need for CiphertextLength

0. Problem Statement

Arrow PME expects encryption algorithms to calculate the ciphertext length (i.e. size of the encrypted payload) *before* the actual encryption takes place. This is feasible today because the single encryption algorithm implemented in Arrow implementation, AES (`AesEncryptor`) makes it easier to calculate the ciphertext length, based purely on the size of the plaintext - ciphertext length is equal to the size of the plaintext plus the size of a few other well known fields/constants.

We're now extending Arrow to enable Parquet encryption via external encryptors. We need a solution to handle the limitation where not every encryption algorithm allows to know the ciphertext length ahead of time.

1. Background

1.1 Buffer Management for Encryption

Within Apache Arrow PME, encryption occurs at a column level. The code that performs the writing of encrypted Parquet columns contains logic that (a) obtains from the encryptor the estimated ciphertext length, and (b) based on that size, allocates (or resizes) a buffer (essentially a byte array) to be passed to the encryptor for the results of encryption to be stored in. Parquet's `ColumnWriter` delegates the writing/encryption to `PageWriter` objects.

The relevant code within the `PageWriter` class looks like follows.

```
if (data_encryptor_.get()) {
  UpdateEncryption(encryption::kDictionaryPage);
  PARQUET_THROW_NOT_OK(encryption_buffer_->Resize(
      data_encryptor_->CiphertextLength(output_data_len), false));
  output_data_len =
      data_encryptor_->Encrypt(compressed_data->span_as<uint8_t>(),
         encryption_buffer_->mutable_span_as<uint8_t>());
      output_data_buffer = encryption_buffer_->data();
}
```

The `encryption_buffer_` reference is an object of the interface type `ResizableBuffer`, which is ultimately fulfilled by `PoolBuffer`. `ResizableBuffer::mutable_span_as()`

will return its underlying byte array. `ResizableBuffer` provides utility methods for re-adjusting the size of the underlying byte array (which may potentially change upon a resize operation)

As seen, the `ColumnWriter` (just one of the places where encryption may be requested) invokes two methods within the encryptor: (a) `CiphertextLength`, and (b) `Encrypt`. These are the complete signatures for these two methods of `Encryptor`:

```
int32_t CiphertextLength(int64_t plaintext_len) const;

int32_t Encrypt(::arrow::util::span<const uint8_t> plaintext,
                ::arrow::util::span<uint8_t> ciphertext);
```

Note that the input to `CiphertextLength` is a *length* parameter - the length of the plaintext - as opposed to the plaintext itself. This means that the first time that an encryptor sees the plaintext payload is during the call to `Encrypt()`, which happens after invoking `CiphertextLength()`.

1.2 Arrow and External Encryptors

In order understand the proposed solutions below, we need to provide a bit of context on that how external encryptors will be incorporated during the Arrow PME Extension project (fully documented here):

- We will define a new interface, DataBatchProtectionAgent (DBPA) for external encryption/decryption agents.
- DBPAs will be unaware of Arrow-related concepts.
- DBPAs will be consumed by Arrow via shared libraries (e.g. *.so files)
- The call to `DBPA::Encrypt()` will return an `EncryptionResult` object.
- The EncryptionResult object contains, among other things
  - `byte[] dbpa_ciphertext`, the buffer with the encrypted payload
  - `int32 ciphertext_length`, the size of the encrypted payload
- We will create a new 'adapter' class in the Arrow codebase (`ExternalDBPAEncryptorAdapter`, working name). From an Arrow perspective, this class will behave as a regular encryptor, however all the encryption/decryption functionality will be forwarded/delegated to an underlying DBPA.
- During `ExternalDBPAEncryptorAdapter::Encrypt(plaintext, encrypted_out_buf)`:
  1. `ExternalDBPAEncryptorAdapter` calls `DBPA::Encrypt(plaintext)`, upon which the DBPA returns a `EncryptionResult` object (`encryption_res`)
     a. The allocation of the `EncryptionResult.dbpa_ciphertext` buffer is an internal decision of DBPA, independent of Arrow's memory management mechanisms.

2. Based on `encryption_res.ciphertext_length` `ExternalDBPAEncryptorAdapter` resizes, `encrypted_out_buf`
3. `ExternalDBPAEncryptorAdapter` copies the contents of `encryption_res.dpba_ciphertext` into `encrypted_out_buf`
4. `ExternalDBPAEncryptorAdapter` releases (i.e. invokes the destructor of) `encryption_res.`
    a. The decision of what happens to `EncryptionResult.dbpa_buffer` upon destruction of an `EncryptionResult` object is internal to the DBPAgent. The agent may decide to simply release the buffer's memory, but it may also decide to return the buffer to an agent-internal memory pool, etc.
5. `ExternalDBPAEncryptorAdapter::Encrypt()` returns.

2. Potential Solutions

High-level, there are two dimensions for encryptors unable to predict ciphertext length, specifically how will they:

A. Signal to Arrow that ciphertext length cannot be provided
B. Pass the encrypted payload back to Arrow

Note: "external encryptors" in this section refer to Arrow-internal encryptors or encryptor-like entities, such as the `ExternalDBPAEncryptorAdapter` adapter class mentioned above. In other words, a DataBatchProtectionAgent is not considered an external encryptor (in the context of this document).

2.1. Signaling the inability to provide ciphertext length

2.1.1. [IMPLEMENTED] Add a "can provide ciphertext length?" signal to the Encryptor.

Requires Modification to Arrow's Codebase? ***Yes.***
Requires Modification to existing encryptor? ***Yes (minimal).***

Under this solution, the encryptor will signal Arrow, via a new flag or similar, whether it can provide a value for ciphertext length. If `flag == true`, then Arrow will follow the existing workflow. If `flag == false`, then Arrow will accept that a proper-sized buffer cannot be accommodated pre-encryption, and will use other mechanisms to obtain the encrypted data. This flag can be provided via a new method (e.g. `CanProvideCiphertextLength()`), or alternatively we could make more complex the return type of the existing `CiphertextLength()` method to provide the signal.

Pros:

- Both the interface and the implementation of this approach are clean and simple.
- Retrofitting the existing Arrow code does not represent a significant amount of work.

Cons:

- Requires modifying the Arrow codebase.

2.1.2. Not signaling, and using a heuristic instead

Requires Modification to Arrow's Codebase? ***No***
Requires Modification to existing encryptor? ***No***

In this approach, there is no signal that an encryptor is unable to calculate ciphertext length. Instead, the encryptor will decide (during the call to `CiphertextLength()`) based on a simple heuristic (e.g. multiply the plain text length by 1.5 or 2.0)

Pros:

- No modification of existing Arrow interfaces/code.

Cons:

- Hard to come up with a heuristic
- Heuristic can be inaccurate
    - Too high, and memory being wasted
    - Too low, and encryption cannot take place.

2.2. Passing the encrypted payload back to Arrow

2.2.1. [IMPLEMENTED] Allowing the external encryptor to resize the buffer

Requires Modification to Arrow's Codebase? ***Yes.***
Requires Modification to existing encryptor? ***Yes***

Under this solution, the encryptor will receive a reference to `encryption_buffer_`. The encryptor can decide internally whether to resize `encryption_buffer_` either *before* or *after* performing encryption.

The signature of `Encryptor::Encrypt()` would be modified to:

```
int32_t Encrypt(::arrow::util::span<const uint8_t> plaintext,
                std::shared_ptr<::arrow::ResizableBuffer> encryption_buffer_
    );
```

Pros:

- No waste of memory

- No need for heuristics
- If desired, this approach can also simplify the Encryptor interface as now the encryptors that are able to, can calculate the ciphertext length internally and perform the resizing of the buffer directly.

Cons:
- Requires modification of existing Arrow code
- Yielding control of `encryption_buffer_` can lead to intentional and/or unintentional abuse.

2.2.2. Receiving data block from encryptor

Requires Modification to Arrow's Codebase? ***Yes.***
Requires Modification to existing encryptor? ***Yes***

In this approach, the contract with encryptors is modified so that they allocate the memory required for encryption, and the memory block (i.e. buffer) is returned once the call to `Encryptor::Encrypt()` completes. The caller (`PageWriter` in this case) is responsible for de-allocating the memory block once it's not needed.

Pros:
- Avoids exposing Arrow internal mechanisms (Buffers) to encryptors

Cons:
- Requires modification of existing Arrow code. This change is more significant than in other options, as it becomes necessary to manage the buffer returned by the encryptor.
- Inefficient (because of memory allocation/deallocation overhead) where encryptors (such as the existing AES_TODO encryptor) have the ability to calculate ciphertext length.
- Also inefficient (because of memory allocation/deallocation overhead) across calls to the same encryptor (which is re-used at the column level).

2.2.3. Assume a 'max' ciphertext length.

Requires Modification to Arrow's Codebase? ***Yes, minimal.***
Requires Modification to existing encryptor? ***No***

In this approach, for those encryptors unable to estimate ciphertext length, Arrow will estimate a high value for the size of the memory buffer.

Pros:
- Simple
- Minimal modification to Arrow codebase.

Cons:
- Very wasteful (memory usage)
- Can cause errors because of excessive memory usage, especially when writing tables with a large number of encrypted columns.

3. Changes to the Arrow codebase

To summarize the previous section, in order to support encryptors that cannot provide ciphertext length ahead of time, we're proposing to (a) instrument the encryptor with a `CanProvideCiphertextLength()` function, and (b) for encryptors which cannot provide ciphertext length, we will create a new `EncryptWithManagedBuffer()` function.

3.1 Interface changes

The `Encryptor` interface (in `parquet/encryption/internal_file_encryptor.h`) will be extended with these two methods

```
int32_t EncryptWithManagedBuffer(
    ::arrow::util::span<const uint8_t> plaintext,
    ::arrow::ResizableBuffer* encryption_buffer);

bool CanCalculateCiphertextLength() const;
```

Encryptors that cannot provide ciphertext length should throw an exception if the already-existing `Encrypt()` method is invoked. Conversely, encryptors that can provide ciphertext length should throw an exception when the `EncryptWithManagedBuffer()` method is invoked.

3.2 Code Changes

Below are the locations in the Arrow codebase that need to be updated to understand the new concepts from Encryptor (`CanCalculateCiphertextLength()` and `EncryptWithManagedBuffer()`)

—- —- —- —-

`parquet/column_writer.cc`

Need to be modified in two places

- `SerializedPageWriter::WriteDictionaryPage()` (around line 306)
- `PageWriter::WriteDataPage()` (around line 398)

—- —- —- —-

`parquet/metadata.cc`

Need to be modified in two places:

- `FileMetadataImpl::WriteTo()` (around line 860)
- `ColumnChunkMetaDataBuilderImpl::Finish()` (around line 1735)

—- —- —- —-

parquet/thrift_internal.h

Need to be modified in two places:

- `ThriftSerializer::SerializeEncryptedObj()` (around line 689)

Each code location has some nuance, but the general idea is to ensure that these locations follow this pattern:

```
C/C++
   //example on how to make use of
   // canProvideCiphertextLength() and
   // EncryptWithManagedBuffer()

   if (encryptor->canProvideCiphertextLength()) {
      // Use the existing workflow for encryptors that can provide
      // precise ciphertext length
      auto cipher_buffer =
          //sometimes, the buffer will be already allocated.
          AllocateBuffer(encryptor->pool(),
                         encryptor->CiphertextLength(plaintext_len));
      cipher_buffer_len =
          encryptor->Encrypt(out_span,
                             cipher_buffer->mutable_span_as<uint8_t>());
    } else {
      // Use the new workflow for encryptors that cannot provide
      // precise ciphertext length

       // sometimes, the buffer will be already allocated.
      auto cipher_buffer = AllocateBuffer(encryptor->pool(), 0);
      cipher_buffer_len =
          encryptor->EncryptWithManagedBuffer(out_span, cipher_buffer.get());
    }
```

3.3 ExternalDBPAEncryptorAdapter - example implementation

This is an example of how `ExternalDBPAEncryptorAdapter` will look internally (simplified for illustrative purposes)

C/C++

```
int32_t ExternalDBPAEncryptorAdapter::Encrypt(
   span<const uint8_t> plaintext,
   span<uint8_t> ciphertext)
{
   throw std::exception("Encrypt not supported. Must use
EncryptWithManagedBuffer");
}

bool ExternalDBPAEncryptorAdapter::canProvideCiphertextLength() {
   return false;
}

int32_t ExternalDBPAEncryptorAdapter::EncryptWithManagedBuffer(
   span<const uint8_t> plaintext,
   ::arrow::ResizableBuffer* encryption_buffer) {

   //this was loaded during init().
   EncryptionResult* encrypt_res = dbpagent_instance_->encrypt(plaintext);

   if (!encrypt_res) {
     throw std::exception("failed to encrypt via dbpagent_");
   }

   int32_t actual_size = encrypt_res->ciphertext_length_;

   //resize the Arrow-owned buffer
   encryption_buffer->Resize(actual_size, false); //do not trim to fit.

   // Copy the encrypted data into encryption_buffer
   std::memcpy(
      encryption_buffer->mutable_data(),
      encrypt_res->ciphertext,
      actual_size);

   return actual_size;
}
```

4. Decryption Workflow

The changes in the decryption workflow are similar to those in encryption. However, there is a nuance: currently, decryptors implement both `plaintextLength()` (as an equivalent to `ciphertextLength()` in the encryption workflow).

However, it is safe to assume that if a decryptor can calculate the length of its plaintext ahead of time, it can also calculate the length of its ciphertext as well. For this reason, the `Decryptor` interface (in `parquet/encryption/internal_file_deryptor.h`) will be extended with these two methods

```
int32_t DecryptWithManagedBuffer(
    ::arrow::util::span<const uint8_t> ciphertext,
    ::arrow::ResizableBuffer* decryption_buffer);

bool CanCalculateLengths() const;
```

As with the Encryptor, the calls to decrypt should be preceded by a check for whether the decryptor can calculate the lengths ahead of time. If so, the regular `Decrypt()` call is used, and the `DecryptWithManagedBuffer()` option is used otherwise.

Decryptors that cannot calculate the length, like our `ExternalDBPADecryptorAdapter`, should throw an exception if either `CiphertextLength()` or the already-existing `Decrypt()` method is invoked. Likewise, the decryptors that can calculate the lengths should throw an exception if `DecryptWithManagedBuffer()` is called.

5. Memory Utilization Concerns

During the exploratory phases of this work, some concerns regarding memory usage were brought up. In particular, there was the concern on whether passing the managed buffer to the encryptors would result in memory leaks or some type of abuse of memory.

Because (1) the managed buffer is *only* to be used by Arrow-internal components (i.e. `ExternalDBPAEncryptorAdapter`), (2) the buffer passed from the DataBatchProtectionAgent::Encrypt() is also to be managed by `ExternalDBPAEncryptorAdapter` we can guarantee that memory will not be abused.

For added peace of mind, we developed a test where we exercised the Arrow memory pool being passed to and modified by `ExternalDBPAEncryptorAdapter`. We then used **Valgrind** to successfully verify that memory management worked as expected, and no memory leaks were caused by passing the memory buffer handle around.

6. Appendix: Raw Notes

Buffer is of type ResizableBuffer
https://github.com/apache/arrow/blob/main/cpp/src/arrow/buffer.h#L494

Size must be a multiple of 64
https://github.com/apache/arrow/blob/main/cpp/src/arrow/memory_pool.cc#L906-L914
https://github.com/apache/arrow/blob/main/cpp/src/arrow/memory_pool.cc#L947-L956

Size must be 64-byte aligned
https://github.com/apache/arrow/blob/main/cpp/src/arrow/memory_pool.h#L117-L120

—--
Page Writer is used within ColumnWriter
Two implementations of PageWriter

- SerializedPageWriter
- BufferedPageWriter
    - Delegates behavior to SerializedPageWriter
- Both types support encryption.

We do not want to pass a handle of the memory buffer from Arrow to DataBatchProtectionAgent. This would be tying up semantics/impl details of Arrow to the DBPA